Faculty of Engineering and Technology

Department of Computer Science

Artificial Intelligence - COMP338

Project Report No.2

Classifying News Headlines Using Decision Tree and Logistic Regression

---

Prepared by:

Students Name:	**IDs:**
Rahaf Badwan	1220654
Aya Abdalmuti	1212074

Instructor: Dr. Mohammad Helal **and** Dr. AbdelRahman Hamrsha

Semester: 2025/2026.

Abstract

This project aims to classify news headlines into four predefined categories using the AG News dataset. The methodology involves text preprocessing and feature extraction via TF-IDF vectorization. Two machine learning models, Logistic Regression and Decision Tree, were implemented and evaluated based on Accuracy, Precision, Recall, and F1-score. The experimental results demonstrate that Logistic Regression outperformed the Decision Tree model, showing it works better for text data with many features.

Table Of Contents

Abstract 2

1. Introduction 5

2. Dataset Overview 5

3. Data Preprocessing 6

4. Feature Extraction (TF-IDF) 6

5. Data Splitting 7

6. Models & Training 7

6.1 Logistic Regression 7

6.2 Decision Tree 8

7. Experimental Setup 8

8. Code Implementation 9

8.1 Text preprocessing 9

8.2 Loading the Dataset 9

8.3 TF-IDF Feature Extraction 10

8.4 Data Splitting and TF-IDF Feature Transformation 11

8.5 Model Training (Logistic Regression & Decision Tree) 12

8.6 Model Evaluation and Performance Metrics 12

8.7 Results Visualization and Output Files 13

9. Evaluation Metrics and Results 14

9.1 Metrics Explanation 14

9.2 Results of Logistic Regression and Decision Tree 14

9.3 Confusion Matrices 15

9.4 Decision Tree 18

10. Discussion 19

11.Conclusion 20

12.References 21

Table of Figure

Figure 1 : Text preprocessing. 9

Figure 2: Loading the Dataset. 9

Figure 3 : TF-IDF Feature Extraction..... 10

Figure 4:Train / Validation / Test Split and TF-IDF Feature Transformation 11

Figure 5:Training Logistic Regression and Decision Tree Models..... 12

Figure 6:Model Evaluation Using Accuracy, Precision, Recall, F1-score, and Confusion Matrix 12

Figure 7:Performance Comparison and Decision Tree Visualization 13

Figure 8: Bar Chart of Model Performance Comparison. 15

Figure 9: Confusion matrix showing the classification results of the Logistic Regression model on the test dataset. 16

Figure 10: Confusion matrix showing the classification results of the Decision Tree model on the test dataset. 17

Figure 11: Visualization of the Decision Tree model used for news headline classification based on TF-IDF features..... 18

List of Table

Table 1: Overall Accuracy, Precision, Recall, and F1-score for Each Model. 14

1. Introduction

Text classification is an important task in Natural Language Processing (NLP) that automatically assigns categories to text based on its content. With the large growth of digital information, text classification helps organize and retrieve big datasets efficiently.

This technology is used in many applications such as spam detection, sentiment analysis, and topic categorization. In digital journalism, news classification is very important. Since many news articles are published every day, automatically sorting headlines into categories like World, Sports, Business, and Science/Technology makes it easier to recommend content and analyze it systematically.

The main goal of this project is to develop and evaluate machine learning models that classify news headlines using the AG News dataset. Two algorithms are implemented and compared: Logistic Regression and Decision Tree. The project covers the complete process, including text preprocessing, feature extraction with TF-IDF, and performance evaluation using metrics such as Accuracy and F1-score. This comparison shows the strengths and weaknesses of each model when classifying short news headlines.

2. Dataset Overview

The dataset used in this project is the **AG News Dataset**, which is publicly available on HuggingFace. It contains short news headlines collected from various news sources and is categorized into four classes: **World, Sports, Business, and Science/Technology**.

The training set contains **120,000 samples**, while the test set contains **7,600 samples**. Each category is represented evenly, which allows for a **balanced evaluation** of the classification models. The balanced nature of the classes ensures that the models (Logistic Regression & Decision Tree) are trained fairly without bias toward any specific news category.[1]

3. Data Preprocessing

Before applying machine learning algorithms, it is important to preprocess the text data to improve the performance of the models. In this project, the following preprocessing steps were applied to the news headlines:

1. **Lowercasing:** All text was converted to lowercase to make the words consistent.
2. **Removing URLs:** Any links or web addresses in the text were removed, as they do not provide useful information for classification.
3. **Removing symbols and punctuation:** Special characters, punctuation marks, and other irrelevant symbols were removed to keep only the meaningful text.

Preprocessing is essential because machine learning models work better with clean and consistent data. Removing noise from the text reduces confusion for the model, improves feature extraction, and increases the overall accuracy of the classification.

4. Feature Extraction (TF-IDF)

After preprocessing the text, the next step is to convert the news headlines into numerical features that can be used by machine learning models. In this project, TF-IDF (Term Frequency-Inverse Document Frequency) was used for feature extraction. TF-IDF is a method that reflects how important a word is in a document compared to all other documents in the dataset. Words that appear frequently in a single document but rarely in other documents get higher scores, which helps the model focus on meaningful words for classification.

TF-IDF was chosen because it is simple and effective for text data, works well with sparse high-dimensional data like news headlines, and emphasizes words that are important for distinguishing between different categories. The TF-IDF vectorizer in this project was

configured with ngram_range = (1, 2), which considers both single words (unigrams) and consecutive word pairs (bigrams), max_features = 20,000 to limit the vocabulary for efficiency, min_df = 2 to ignore words that appear in very few documents, and max_df = 0.9 to ignore words that appear in almost all documents, as they are usually not informative. These settings help the models focus on the most useful features while reducing noise and memory usage.[3]

5. Data Splitting

After preprocessing and converting the headlines into numerical features using TF-IDF, the dataset was split into a **training set** and a **test set**. The training set is used to train the models, while the test set is reserved for evaluating their performance. This ensures that the models are evaluated on unseen data, providing a fair assessment of their classification capabilities.

6. Models & Training

6.1 Logistic Regression

Is a supervised machine learning algorithm used for classification tasks. It works by predicting the probability that a given input belongs to each class and then assigning the input to the class with the highest probability. Logistic Regression is suitable for text classification because it performs well with high-dimensional and sparse data, which is typical when using TF-IDF features. It can efficiently handle large vocabularies and provides probabilistic outputs, which help in understanding the model's predictions. In this project, the Logistic Regression model was trained with a maximum of 2000 iterations (max_iter = 2000) to allow the algorithm enough time to converge, and a fixed random state (random_state = 42) was used to ensure reproducibility of results.[4]

6.2 Decision Tree

Is a supervised machine learning algorithm that classifies input data by recursively splitting it based on feature values. Each internal node represents a feature, each branch represents a decision, and each leaf node represents the predicted class. The tree selects the feature that best separates the classes at each split, and this process continues until a stopping condition is reached, such as a maximum depth or a minimum number of samples per node. Decision Trees may perform worse with TF-IDF features because high-dimensional and sparse data makes it difficult for the tree to find effective splits. Additionally, trees can easily overfit to the training data when there are many features, resulting in poor generalization on test data. In this project, the Decision Tree was trained with a maximum depth of 3 (max_depth = 3) to prevent overfitting, a minimum of 10 samples required to split a node (min_samples_split = 10), a minimum of 5 samples per leaf node (min_samples_leaf = 5), and a fixed random state (random_state = 42) to ensure reproducibility.[5]

7. Experimental Setup

The classification models in this project were implemented using Python and the Scikit-learn library. The dataset was split into training and test sets. The training set was used to train the models, while the test set was used to evaluate their performance on unseen data. [2]

Both Logistic Regression and Decision Tree models were trained on the TF-IDF features extracted from the news headlines. Weighted metrics were used for Accuracy, Precision, Recall, and F1-score because the dataset contains multiple classes. Using weighted averages ensures that the performance of each class is properly considered in the final results. [3]

8. Code Implementation

8.1 Text preprocessing

```python
def clean_text(text: str) -> str:
    """
    Basic cleaning:
    - lowercase
    - remove urls
    - keep letters/numbers/spaces
    - collapse extra spaces
    """
    if text is None:
        return ""
    text = text.lower()
    text = re.sub(r"http\S+|www\.\S+", " ", text)          # remove URLs
    text = re.sub(r"[^a-z0-9\s]", " ", text)               # remove punctuation/special chars
    text = re.sub(r"\s+", " ", text).strip()               # collapse spaces
    return text
```

Figure 1 : Text preprocessing.

The clean text function preprocesses raw news headlines by converting text to lowercase, removing URLs and punctuation, and collapsing extra spaces. This ensures that the text is clean and consistent for machine learning.

8.2 Loading the Dataset

```python
def load_ag_news_dataframe() -> pd.DataFrame:
    """
    Loads AG News from HuggingFace and returns a single DataFrame
    with columns: text, label, split
    """
    ds = load_dataset("ag_news")  # has train/test splits
    train_df = pd.DataFrame(ds["train"])
    test_df = pd.DataFrame(ds["test"])
    train_df["split"] = "train"
    test_df["split"] = "test"
    df = pd.concat([train_df, test_df], ignore_index=True)

    # Clean text
    df["text"] = df["text"].astype(str).apply(clean_text)
    return df
```

Figure 2: Loading the Dataset.

This function loads the AG News dataset from Hugging Face, combines training and test sets into a single Data Frame, and applies the text preprocessing. It also labels each row as train or test.

8.3 TF-IDF Feature Extraction

```
def build_vectorizer():
    """
    TF-IDF Vectorizer parameters (mention in report):
    - ngram_range: (1,2) -> unigrams + bigrams
    - max_features: limit vocabulary for speed/memory
    - min_df: ignore very rare terms
    - max_df: ignore very common terms
    """
    return TfidfVectorizer(
        ngram_range=(1, 2),
        max_features=20000,
        min_df=2,
        max_df=0.9
    )
```

Figure 3 : TF-IDF Feature Extraction.

The build vectorizer function defines the TF-IDF vectorizer, which converts text into numerical features. It uses unigrams and bigrams, limits vocabulary size, and ignores very rare or very common terms.

8.4 Data Splitting and TF-IDF Feature Transformation

```
# Use HuggingFace official splits
train_df = df[df["split"] == "train"].copy()
test_df = df[df["split"] == "test"].copy()

X_train_full = train_df["text"].values
y_train_full = train_df["label"].values

X_test = test_df["text"].values
y_test = test_df["label"].values

# Validation split from train
X_train, X_val, y_train, y_val = train_test_split(
    X_train_full, y_train_full,
    test_size=0.1,
    random_state=RANDOM_STATE,
    stratify=y_train_full
)

# TF-IDF
vectorizer = build_vectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_val_vec = vectorizer.transform(X_val)
X_test_vec = vectorizer.transform(X_test)
```

Figure 4:Train / Validation / Test Split and TF-IDF Feature Transformation

The dataset can then be split between training and test sets based on the AG NEWS split. In addition to this, the validation set (10% of the training set) can be obtained using stratified sampling. Subsequently, the text could be converted to numerical features using TF-IDF. In this approach, the fitting of the vectorizer can result only on the training set, even though the validation and test sets are transformed based on the vocabulary.

8.5 Model Training (Logistic Regression & Decision Tree)

```python
# Model 1: Logistic Regression
lr = LogisticRegression(
    max_iter=2000,
    random_state=RANDOM_STATE
)
lr.fit(X_train_vec, y_train)

# Validation check
val_pred_lr = lr.predict(X_val_vec)
val_acc_lr = accuracy_score(y_val, val_pred_lr)
print(f"\nLogistic Regression Validation Accuracy: {val_acc_lr:.4f}")

# Model 2: Decision Tree
#
dt = DecisionTreeClassifier(
    random_state=RANDOM_STATE,
    max_depth=3,
    min_samples_split=10,
    min_samples_leaf=5
)
dt.fit(X_train_vec, y_train)
```

Figure 5:Training Logistic Regression and Decision Tree Models

Two classification models will be trained with TF-IDF features: a logistic regression model and a decision tree.

Logistic Regression is used as a linear base model, whereas the Decision Tree model uses a lower depth for training to prevent overfitting.

8.6 Model Evaluation and Performance Metrics

```python
# Evaluate on TEST set
results = []
results.append(evaluate_model(lr, X_test_vec, y_test, "Logistic Regression", output_dir, show_plots=SHOW_PLOTS))
results.append(evaluate_model(dt, X_test_vec, y_test, "Decision Tree", output_dir, show_plots=SHOW_PLOTS))
```

Figure 6:Model Evaluation Using Accuracy, Precision, Recall, F1-score, and Confusion Matrix

The trained models are used to test the test dataset. For evaluating the models, the accuracy, precision, recall, and weighed F-1 Score are considered.

A confusion matrix will also be created to visualize the classification results on all news categories.

8.7 Results Visualization and Output Files

```
# Metrics comparison plot (save + display)
metrics_plot_path = plot_metrics_comparison(results_df, output_dir, show_plots=SHOW_PLOTS)

# Decision tree plot (save + display)
feature_names = vectorizer.get_feature_names_out()
tree_plot_path = save_decision_tree_plot(dt, feature_names, output_dir, show_plots=SHOW_PLOTS)
```

Figure 7:Performance Comparison and Decision Tree Visualization

Model performance is visualized using a comparison bar chart for evaluation metrics.
In addition, the trained Decision Tree is visualized to illustrate the learned decision rules and feature importance.

9. Evaluation Metrics and Results

9.1 Metrics Explanation

To evaluate the models, we used Accuracy, Precision, Recall, and F1-score. Because the dataset has multiple categories (World, Sports, Business, Sci/Tech), we calculated the metrics using weighted averages, which makes sure each class contributes proportionally to the final score.

- **Accuracy** shows the overall percentage of correctly classified news headlines.
- **Precision** shows how many headlines predicted for a class are actually correct.
- **Recall** shows how many of the actual headlines for a class the model successfully found.
- **F1-score** combines Precision and Recall into a single measure, balancing quality and coverage.

9.2 Results of Logistic Regression and Decision Tree

Both models have been tested on the test set. Logistic Regression outperformed the Decision Tree in all metrics. The reason is that Logistic Regression works better when the number of features is very large and sparse, like TF-IDF features in text processing, while Decision Tree has difficulty finding optimal splits with this kind of data, hence it suffers from overfitting a lot.

Table 1: Overall Accuracy, Precision, Recall, and F1-score for Each Model.

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	**0.92**	**0.92**	**0.92**	**0.92**
Decision Tree	**0.36**	**0.70**	**0.36**	**0.31**

The overall average of the values provides an overview of how the model is performing overall, but does not guarantee that the values are performing equally well on a class basis. For example, Decision Tree results demonstrate an uneven performance level between precision and recall, suggesting performance level variation between classes, despite average results indicating

otherwise.

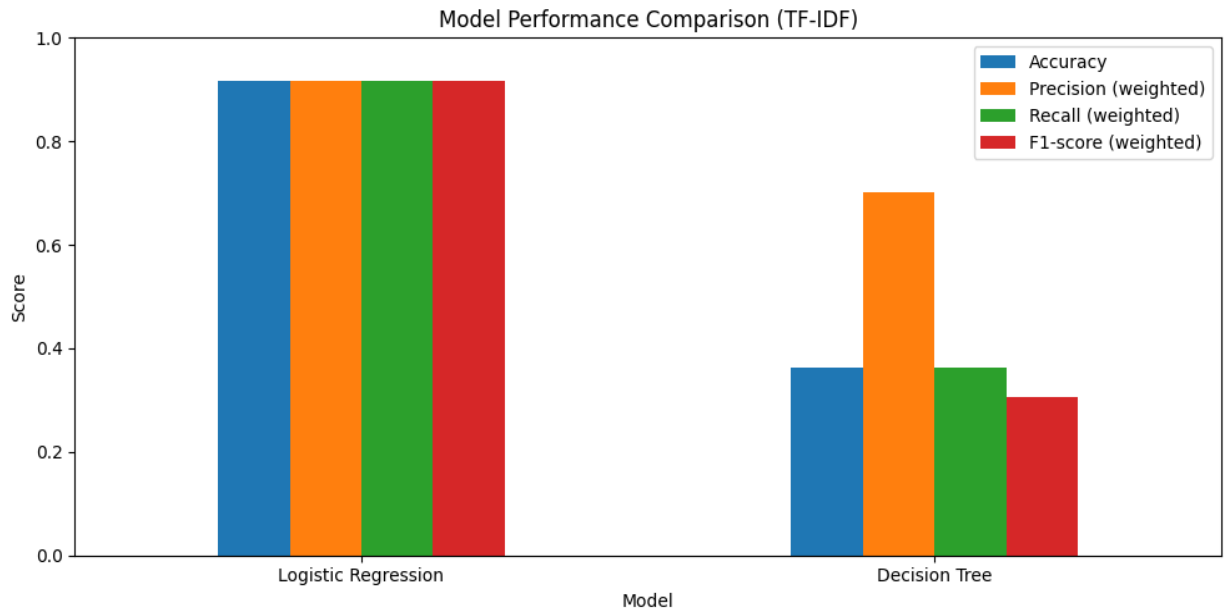

Figure 8: Bar Chart of Model Performance Comparison.

The bar chart illustrates the performance of Logistic Regression and Decision Tree on the test set. Logistic Regression achieves higher scores across all metrics, showing its strength in handling high-dimensional, sparse TF-IDF features. The Decision Tree performs worse, likely due to difficulty finding optimal splits and overfitting. This visual clearly highlights the differences in classification performance between the two models.

9.3 Confusion Matrices

Confusion matrices provide a detailed view of the classification performance of each model by showing the number of correct and incorrect predictions for each class. They help identify which categories are classified well and which classes are more likely to be confused.[7]

- **Confusion Matrix for Logistic Regression Model**

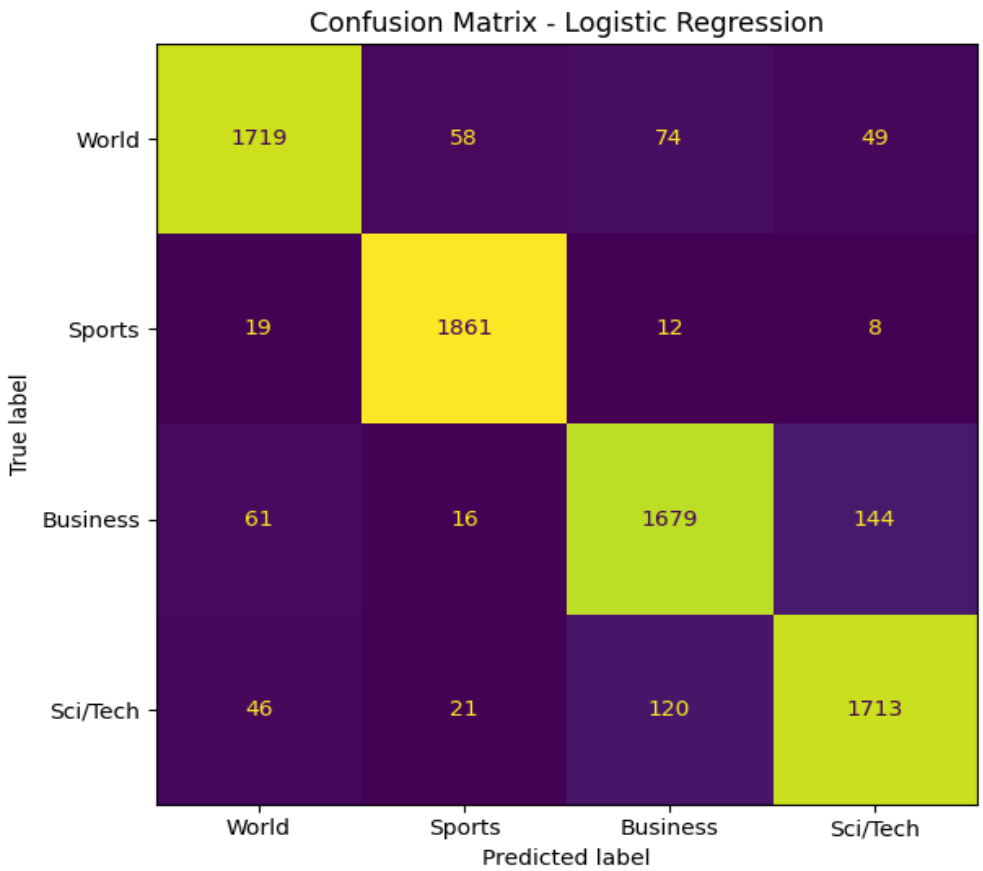

Figure 9: Confusion matrix showing the classification results of the Logistic Regression model on the test dataset.

The confusion matrix shows that Logistic Regression correctly classified most news headlines across all four categories (World, Sports, Business, and Sci/Tech). The majority of predictions are concentrated along the diagonal, indicating high accuracy and balanced performance across classes.[4]

- **Confusion Matrix for Decision Tree Model**

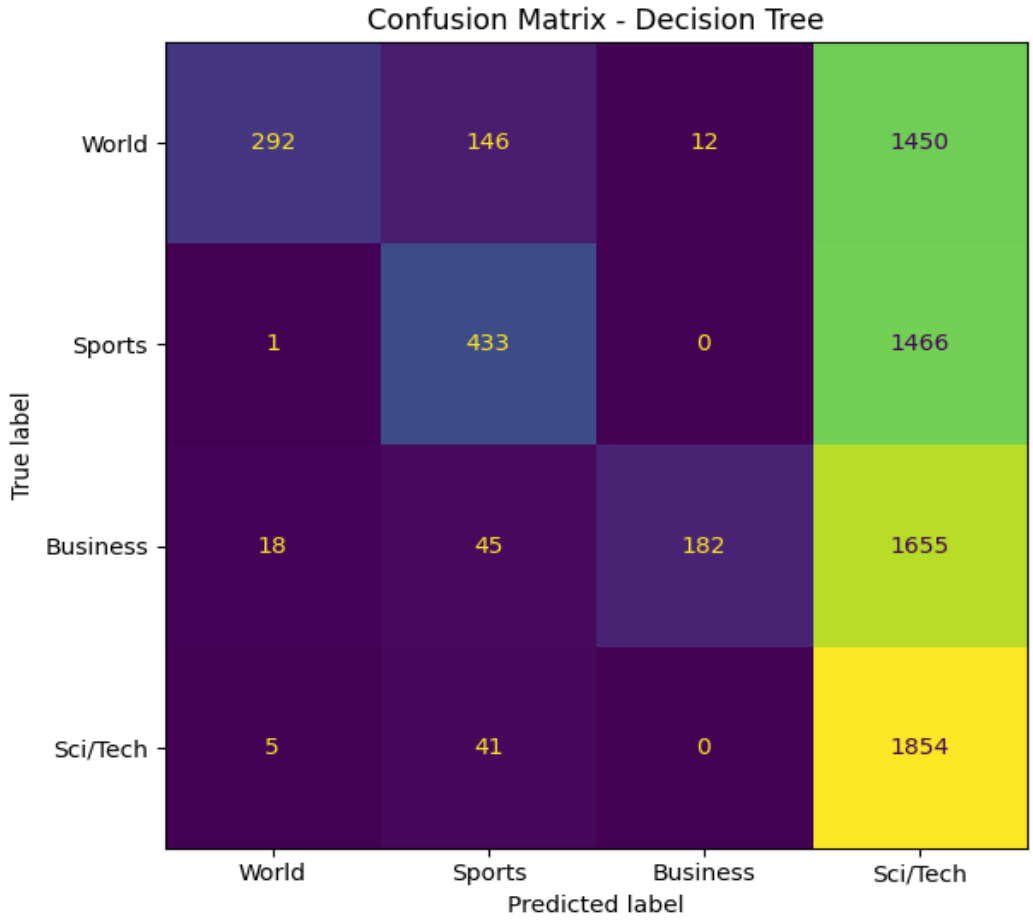

Figure 10: Confusion matrix showing the classification results of the Decision Tree model on the test dataset.

The confusion matrix reveals that the Decision Tree model made more misclassifications compared to Logistic Regression. Some categories, such as Sports and World, were confused with each other, which explains the lower Precision and Recall scores achieved by this model.

9.4 Decision Tree

The Decision Tree visualization shows how the model classifies news headlines by recursively splitting the data based on TF-IDF features. Each internal node represents a feature used for decision-making. Each branch corresponds to a decision rule, and each leaf node indicates the predicted news category.

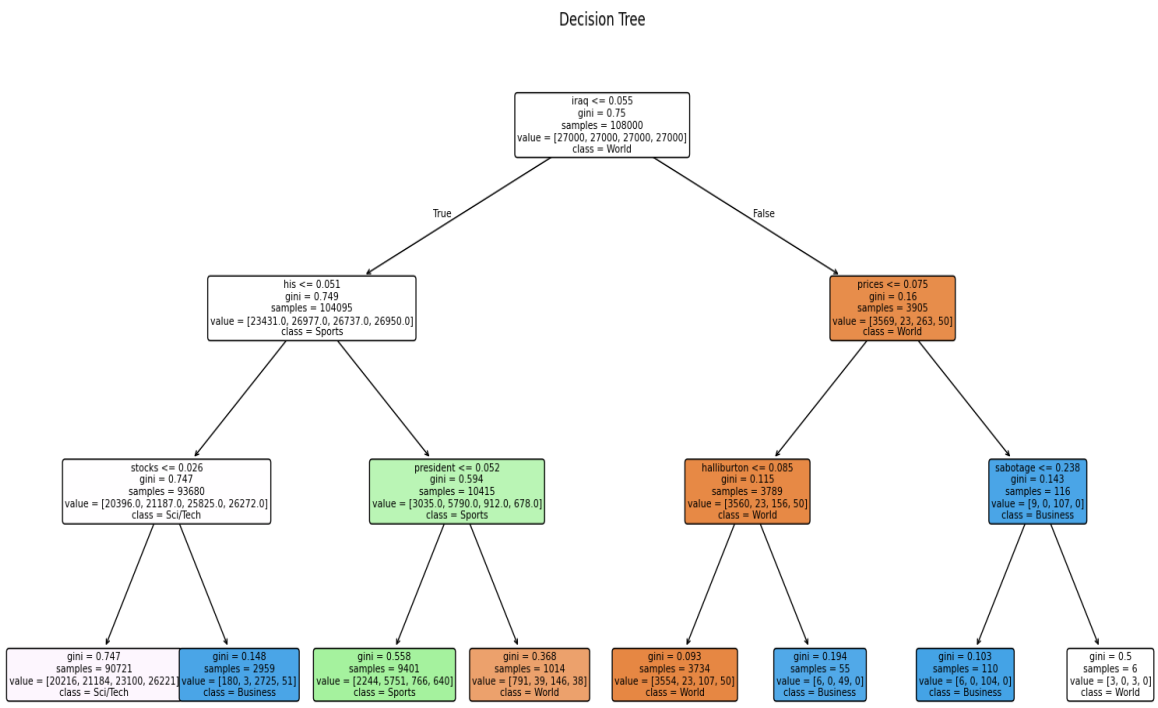

Figure 11: Visualization of the Decision Tree model used for news headline classification based on TF-IDF features.

This visualization helps in understanding how the model makes predictions. However, it also shows a limitation of Decision Trees with high-dimensional data. The tree can grow too complex, learning noise from the training set rather than general trends, which leads to overfitting.[5]

10. Discussion

In this project, we compared the performance of **Logistic Regression** and **Decision Tree** for classifying news headlines.

Logistic Regression achieved higher Accuracy, Precision, Recall, and F1-score compared to the Decision Tree. This is because Logistic Regression works well with **high-dimensional and sparse data**, like the TF-IDF features extracted from the news headlines. It can effectively handle large vocabularies and focuses on the most important words for classification.

On the other hand, the **Decision Tree** performed worse. The tree had difficulty finding effective splits in high-dimensional sparse data and was prone to **overfitting** the training set, which reduced its performance on the test set.

Some additional observations:

> Using **n-grams (unigrams + bigrams)** improved the models' ability to capture meaningful phrases like "White House" or "stock market," which are important for distinguishing between categories.
>
> The model had more difficulty classifying certain categories, such as **Sports vs World**, likely because some headlines contain words that appear in multiple categories.
>
> Weighted metrics ensured that each category contributed fairly to the final evaluation, giving a more accurate picture of overall model performance.

Overall, Logistic Regression is more suitable for **text classification with TF-IDF features**, while Decision Trees may require further techniques like **dimensionality reduction** or **feature selection** to perform better.

11.Conclusion

In this project, we built and evaluated two machine learning models, Logistic Regression and Decision Tree, to classify news headlines into four categories: World, Sports, Business, and Science/Technology.

The results showed that Logistic Regression performed better across all evaluation metrics (Accuracy, Precision, Recall, and F1-score). This is because it handles high-dimensional and sparse TF-IDF features efficiently, while Decision Trees struggled with finding good splits and tended to overfit the training data.

The use of TF-IDF vectorization was crucial for converting text into meaningful numerical features, and weighted metrics ensured a fair evaluation of each class.

For future improvements, the models could benefit from techniques such as word embeddings, dimensionality reduction, or even neural network models to further improve classification accuracy, especially for complex text data.

Overall, this project demonstrates the importance of proper preprocessing, feature extraction, and model selection in text classification tasks.

12.References

[1] AG News Dataset – HuggingFace: https://huggingface.co/datasets/wangrongsheng/ag_news

[2] Scikit-learn Documentation – Text Feature Extraction and Machine Learning Models: https://scikit-learn.org/stable/documentation.html

[3] TF-IDF Explanation – GeeksforGeeks: https://www.geeksforgeeks.org/machine-learning/understanding-tf-idf-term-frequency-inverse-document-frequency/

[4] Logistic Regression – Wikipedia: https://en.wikipedia.org/wiki/Logistic_regression

[5] Decision Tree Classifier – Wikipedia: https://en.wikipedia.org/wiki/Decision_tree_learning

[6] Text Classification – GeeksforGeeks: https://www.geeksforgeeks.org/nlp/what-is-text-classification/

[7] Confusion Matrix - Wikipedia: https://en.wikipedia.org/wiki/Confusion_matrix